MASTER'S THESIS · THIRD MEETING

What happens to AI-generated code after it is submitted?

Studying the fixes made to AI code in real projects

10/19/2026

Last meeting

50+ papers on code changes

Today

We narrowed the focus and checked the papers step by step

We found the gap and started on the data

OUR FOCUS

We study only the fixes of AI code



In scope: fixes

bug · deficiency · missing part



Out of scope

new features · general improvements

Q1

Why was the code changed?

Q2

What was wrong?

Q3

How was it fixed? (the exact change)

STEP 1 · FIND THE CLOSEST PAPERS

~70 → 10

3 criteria to keep a paper:

✓ AI-generated code ✓ later changes ✓ changes are classified

A

After merge

follow AI code in the project

B

Later fix PR

find the PR that fixes an AI PR

C

Before merge

human changes
before accepting

N7 | Only the intent label (corrective...), not how the code was fixed.

The 10 closest papers (Step 1)

Q1 =Why was the code changed? == (Yes) The paper says the reason for the change· / (No) The paper does not give the reason
Q2 = what was wrong? · == (Yes) The paper names the problem in the AI code / (No) The paper does not say what the problem was
Q3 = how was it fixed (exact code change)? == (Yes) The paper describes the exact code change of the fix / (No) The paper does not describe the exact change

#	Group	Paper	Title	Venue	Main idea	Data
1	A – follow AI code after merge	**Rahman & Shihab**	Will It Survive? Deciphering the Fate of AI-Generated Code in Open Source	EASE 2026 (arXiv 2601.16809)	Follows every AI-written line after merge and checks how long it survives and why it is modified later.	AIDev: 201 repos, 5,171 P (3,003 agent), 210,184 lin
2		**Sawada et al.**	To What Extent Does Agent-generated Code Require Maintenance?	EASE 2026	Compares later commits on files created by AI agents vs by humans.	AIDev: 100 repos, 508 AI + human files, 3,238 later c (6 months)
3		**Xia & Miller**	Do These Violent Delights Have Violent Ends? Measuring the Post-Merge Fate of Agentic Code	2026	Tracks agent-written lines for one year after merge and measures the risk of later corrective changes.	Own: 182 repos from the in the Wild" tracker + full history, 1 year
4		**Yu et al.**	Where Is Self-admitted Code Generated by LLMs on GitHub?	2025/2026	Finds code that developers say was made by ChatGPT/Copilot and studies how it is changed later.	Own: ≈1,400 files in 983 G projects
5	B – find the later PR that fixes an AI PR	**Who Finishes the Job?**	Who Finishes the Job? Follow-Up Fixes on AI Agent Pull Requests	2026	Links each merged AI PR to the later PR that fixes it, and finds who made the fix.	AIDev-pop: 6,774 merged PRs + 5,044 human PRs
6		**Khemissi et al.**	Humans Integrate, Agents Fix: How Agent-Authored Pull Requests Are Referenced in Practice	MSR 2026	Studies later PRs that refer to an AI PR: do they fix it or build on it?	AIDev: 33,596 agent PRs; references; 213 labelled
7		**Oukay et al.**	When AI Code Doesn't Stick: An Empirical Study on Reverted Changes Introduced by AI Coding Agents	MSR 2026	Studies AI changes that were later undone (reverted) and why.	AIDev: 33,580 agent PRs, commits; 500 reverting co
8	C – human changes before acceptance	**Watanabe et al.**	On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub	TOSEM	Studies Claude Code PRs: which are accepted, and what humans revise before merge.	Own: 567 Claude Code PR projects (+567 human PRs
9		**Khelifi et al.**	Behind Agentic Pull Requests: An Empirical Study on Developer Interventions in AI Agent-Authored Pull Requests	MSR 2026	Studies what developers do when they intervene in AI PRs.	AIDev: agent PRs vs huma
10		**DECODE (Liang et al.)**	Learning from 53.6K Real-World Developer Edits of AI-Generated Code (DECODE)	2026	Studies how developers edit AI code suggestions in their IDE.	Own: 53.6K developer edi suggestions

Result: **All 10 answer Q1 (why it changed); some answer Q2 (what was wrong); none answers Q3 (how it was fixed). Khelifi and DECODE must be read in full to c**

What they did ?

Paper	What they did	What they count as a "fix"	Their fix result	What the fixes are (do they show the change?)
Who Finishes 2026	Linked each merged agent PR (AIDev-pop) to a **later PR** in the same repo, within 30 days, that touches the same files. An **LLM judge** (Claude Opus) checked each pair, then humans checked a sample.	A later PR labelled `fix` or titled "Revert…" that really fixes the AI PR ("direct fix")	**4.5%** of agent PRs get a verified fix in 30 days. Agent PRs have **1.62×** the odds of human PRs. The fix is by the **same agent 69.6%**, by a **human ~27%**.	Only "direct fix / related / unrelated" and **who** fixed it. **No description of the change.**
Xia & Miller 2026	Followed **every agent line** for 1 year in 182 repos (own line tracking + git blame). Labelled each later change with **rules + an LLM** (MiniMax-M2.7). Ran security scans.	A later commit that changes the line and is **corrective**: bug fix, security fix, revert	About **1.5×** more fixes for agent lines. After 180 days: **4.0%** of agent lines bug-fixed vs **2.7%** for humans.	Fix types: bug fix / security fix / revert. **No description of the change.**
Rahman & Shihab 2026	Tracked **agent lines after merge** (git blame + survival analysis) in AIDev. Labelled later commits from **keywords in the commit message** (Swanson types).	A later commit labelled **corrective**	**26.3%** of later changes to agent code are fixes (vs **23.0%** human). From **13.8% (Cursor)** to **44.4% (Claude Code)**.	Only "corrective". **No description of the change.**
Sawada 2026	Followed later **commits on files created by AI** for 6	A commit labelled `fix`	**11.7%** of later commits on AI	Only the `fix` label. **No description of the**

STEPS 2–3 · CLASSIFY AND CHECK THE DETAIL

Papers give a label, not the change

What papers say

“This is a fix.”

9 classification types: taxonomy, intent, label, bug pattern… all give labels only

#	Classification type	Meaning (simple words)	Papers	Answers which question?	Maintenance?	Use for our thesis
1	Taxonomy	Groups in levels: a big group, then smaller groups inside it (family → sub-group → item).	Oukay, Khelifi (core) · Dovhoshliubnyi*, Wang 2024*, Fan 2023* (added)	Q2 What was wrong? (Oukay, Wang, Fan) · partly Q3 How fixed? (Dovhoshliubnyi = AI's own changes; Fan = fix needed on exercises)	Mostly Yes (problems in AI code). Dovhoshliubnyi = No (AI's own changes, method only).	Model for our own fix taxonomy
2	Intent / action	What the change is FOR: corrective (fix), preventive, perfective (improve), adaptive (environment). Based on Swanson's maintenance types.	Rahman & Shihab, Xia & Miller, Khemissi	Q1 Why was the code changed?	Corrective / fix / revert = Yes · Perfective, adaptive = Check · Constructive / management = No	Filter: keep only fixes
3	Type / label	A short tag from a fixed list, one per commit or PR (e.g. Conventional Commits: fix, feat, refactor, docs, test, perf, style, build, ci, chore).	Sawada, Who Finishes the Job?, Watanabe, DECODE	Q1 Why was the code changed?	fix / test / perf / improving quality = Yes · refactor, docs, style, changing functionality = Check · feat = No	Find fix commits automatically
4	Bug pattern	A kind of mistake in the code (e.g. "missing corner case", "logic error", "hallucinated object").	Yu (core) · outside the 10: Tambon, KNighter	Q2 What was wrong?	Bug fix = Yes · Non-bug = No / Check	Names for "what was wrong"
5	Tool message	Categories given by a tool: static-analysis warnings (SonarQube issue types, Pylint codes) or security weaknesses (CWE IDs).	Not in our 13 · outside: Cynthia, Hasan, Effendi (AWS), Getafix	Q2 What was wrong? (as seen by a tool)	Deficiency → Yes (to confirm with Mohamed if smells/style count)	Detect deficiencies
6	Catalog	A fixed list of change operators (each one = one exact way to change code).	Not in our 13 · outside: HDRepair (mutation operators)	Q3 How was it fixed? (on human code)	Yes (fix operators)	Words for "how it was fixed"
7	Category list	A simple flat list of categories, no levels.	Not in our 13 · outside: Aggarwal, DynaMine, Hora, Change Contracts	Depends on the paper (Q1 or Q2)	Check	Starting list of change kinds
8	Theme	Groups found by reading text (comments, PR descriptions) and coding it by hand (thematic analysis).	Not in our 13 · outside: Abujadallah	Q1 / Q2 (reasons)	Check	Method to label by hand
9	Topic	Groups found automatically by a topic model (e.g. BERTopic) from many texts.	Not in our 13 · outside: Haider (BERTopic)	Q1 / Q2 (review themes)	Check	Method to group many messages

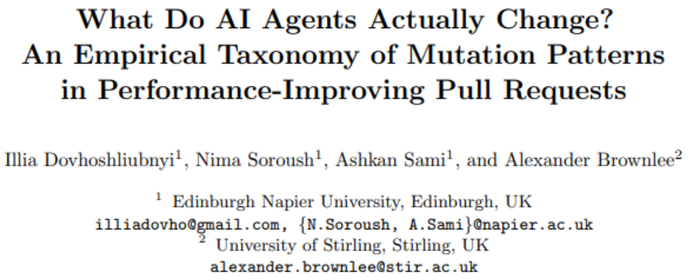


Is there any paper that says exactly how AI code is changed or fixed?

STEPS 4–6 · EXACT CHANGES, TOOLS, EXTRA SEARCH

Two close papers — useful, but not the same

Dovhoshliubnyi 2026



What Do AI Agents Actually Change?
An Empirical Taxonomy of Mutation Patterns
in Performance-Improving Pull Requests

Illia Dovhoshliubnyi[1], Nima Soroush[1], Ashkan Sami[1], and Alexander Brownlee[2]

[1] Edinburgh Napier University, Edinburgh, UK
illiadovho@gmail.com, {N.Soroush, A.Sami}@napier.ac.uk
[2] University of Stirling, Stirling, UK
alexander.brownlee@stir.ac.uk

Wang 2024



Where Do Large Language Models Fail When Generating Code?

Zhijie Wang*
University of Alberta
Edmonton, AB, Canada
zhijie.wang@ualberta.ca

Zijie Zhou*†
University of Illinois
Urbana-Champaign
Urbana, IL, USA
zijiez4@illinois.edu

Da Song*
University of Alberta
Edmonton, AB, Canada
dsong4@ualberta.ca

Yuheng Huang
The University of Tokyo
Tokyo, Japan
yuhenghuang42@g.ecc.u-tokyo.ac.jp

Shengmai Chen
Purdue University
West Lafayette, IN, USA
chen3301@purdue.edu

Lei Ma
The University of Tokyo
University of Alberta
ma.lei@acm.org

Tianyi Zhang
Purdue University
West Lafayette, IN, USA
tianyi@purdue.edu

Repair tools like FixMiner, Getafix, HDRepair: concrete fix patterns, but in human code.

WHAT I FOUND

Why and What never How



THE RESEARCH GAP

We know AI code gets fixed. No study shows how the fix is made in real projects.

This is Q3 — and the focus of the thesis.

WHERE WE ARE NOW · THE PLAN

1 · Collect — now | 2 · Filter | 3 · Describe | 4 · Group

Collecting the datasets shared by the papers

AIDev
AI-agent PRs on GitHub

Agents in the Wild
code after merge, 1 year

Who Finishes
AI PR → later fix PR

DECODE
AI code before / after edit

One notebook per dataset: download → look → first charts → find fixes with our classification table

Also: Yu, Sawada, Watanabe, Dovhoshliubnyi, Khemissi, Wang

1 · AIDev

pull_request.parquet: 100% 57.6M/57.6M [00:03<00:00, 19.1MB/s]

3. First look

```
quick_look(prs)
```

[4] ✓ 0.4s Python

Rows: 71,677 Columns: 14

	id	number	title	body	agent	user_id	user	state	created_at	closed_at	merged_at	repo_id	
0	3369524944	9	feat: Add extensive list of EV charging tools and resources	This pull request resolves the issue by adding an extensive list of over 150...	Google_Jules	161369871	google-labs-jules[bot]	closed	2025-08-30T21:33:07Z	2025-08-30T21:40:30Z	2025-08-30T21:40:30Z	564884602	https://api.github.com/repos/juherr/aw
1	3369556773	10	Add last activity date to GitHub links	This change adds the date of the last activity to GitHub links in the `READM...	Google_Jules	161369871	google-labs-jules[bot]	open	2025-08-30T22:20:29Z	None	None	564884602	https://api.github.com/repos/juherr/aw
2	3205908454	69	Pull request for issue #68	Fixes #68	Google_Jules	161369871	google-labs-jules[bot]	closed	2025-07-06T02:00:56Z	2025-07-06T02:37:38Z	2025-07-06T02:37:38Z	862251163	https://api.github.com/repos/steve0208
				This change ensures that									

	type	missing_%	unique_values	example
id	int64	0.0	71677	[3369524944]
number	int64	0.0	21441	[9]
title	object	0.0	69452	[feat: Add extensive list of EV charging tools and resources]
body	object	0.8	70669	[This pull request resolves the issue by adding an extensive]
agent	object	0.0	6	[Google_Jules]
user_id	int64	0.0	3309	[161369871]
user	object	0.0	3309	[google-labs-jules[bot]]
state	object	0.0	2	[closed]
created_at	object	0.0	71203	[2025-08-30T21:33:07Z]
closed_at	object	8.3	65104	[2025-08-30T21:40:30Z]
merged_at	object	32.4	48365	[2025-08-30T21:40:30Z]
repo_id	int64	0.0	6673	[564884602]
repo_url	object	0.0	6673	[https://api.github.com/repos/juherr/awesome-ev-charging]
html_url	object	0.0	71677	[https://github.com/juherr/awesome-ev-charging/pull/9]

4. Which agents? How many PRs are merged?

```
if "agent" in prs.columns:
    plot_top(prs["agent"], title="PRs per AI agent")
if "state" in prs.columns:
    plot_top(prs["state"], title="PR state")
if "merged_at" in prs.columns:
    print(f"Merged: {prs['merged_at'].notna().mean():.1%}")
```

[5] ✓ 0.2s

